- Paper: Roy, Wetherbee & Fatemi, arXiv:2603.15356 (7 Apr 2026), Cornell + Berkeley.
- Headline claim: a logical gate set $\{X, H, T\}$ on a binomial bosonic qubit that stays *almost* error-transparent mid-gate, using only ordinary linear drives.
- Result to quote: five-fold reduction in logical infidelity conditioned on a photon loss during the gate.
- Three parts today. First the basics, in pictures: what a bosonic qubit is, how gates are done, how one photon loss is corrected, why a loss mid-gate is the problem, and the paper's hardware. Then the paper's theory, up to the three metrics. Then how the pulses are found: GRAPE, the model it runs on, the cost function and the constraints, with the places our replication differs marked "ours". The results, simulated and experimental, are a separate talk.
- Be explicit that this is the paper's work presented by me, plus our simulation replication — not my result.
- Framing sentence to open with: bosonic memories already beat break-even while *idling*. This paper is about what breaks when you try to do a *gate*.

- Start from what everyone in the room knows: a transmon is a qubit because its levels are unequally spaced, so a pulse at ω_q moves only $g \leftrightarrow e$.
- A cavity is the opposite: every step costs the same energy. You cannot single out two levels with a pulse, so instead you *choose* two states that are spread over many photon numbers. That pair is the logical qubit.
- Why go to the trouble. (1) The cavity is a better memory: 180 μs here, near a millisecond in good 3D cavities. (2) Its noise is simple: mostly one photon leaks out, the jump $\hat{a}$. One error type means one thing to correct.
- The extra Fock levels are the redundancy. A qubit with only two levels has no room to signal "an error happened"; the oscillator does.
- The code in this paper: $|0_L\rangle$ is an equal mix of 0 and 4 photons, $|1_L\rangle$ is exactly 2 photons. Point at the green rungs. Both have 2 photons on average — that will matter.
- Name-drop the other codes only so people can place it: cat and GKP use many more photons; the kitten code is the smallest binomial code.

- Two modes matter for control: the cavity (memory) and the transmon (the tool). There is also a readout resonator to read the transmon; it comes back with the hardware slides. Frequencies are from Table I, at the end of this section.
- The cavity on its own is linear: drive it and you only get coherent states. To make gates you need a nonlinearity, and the transmon lends it through χ.
- χ means: the transmon frequency moves by -3.66 MHz for every photon in the cavity. Spectroscopy on the transmon shows one peak per photon number, with height equal to the probability of that photon number. The picture is the cavity in $|0_L\rangle$: peaks only at $n = 0$ and $n = 4$, half each; dashed marks show where the other lines would sit. $|1_L\rangle$ would give one peak at $n = 2$.
- The peaks are 3.66 MHz apart, so a transmon pulse a few hundred ns long can be made to act only when the cavity holds, say, exactly 2 photons. That is the basic conditional operation everything is built from.
- The controls are just two microwave drives, one into each mode, linear in the mode operators. No special nonlinear pumps. This is the "standard toolkit" the paper keeps stressing.

- Start from what everyone has done: a transmon π pulse is one Gaussian, a few tens of ns, calibrated by hand. That works because the transmon's levels are unequally spaced.
- For the cavity no such simple pulse exists: its levels are equally spaced, so the only handle is χ through the transmon. Moving population between Fock states needs both drives working together, in a shape nobody would guess by hand.
- So a computer finds the shape. GRAPE (Gradient Ascent Pulse Engineering) takes a model of the device — the Table I numbers — and the list of what the gate must do, and adjusts the drive value at every ns until the simulated gate is right.
- The output is just a waveform file for each drive, about 1 μs long, played on the same AWG and lines as any other pulse. No extra hardware.
- Model based: the pulse is only as good as the model. If χ or the Kerr is off, the gate is off. That is why the device parameters matter so much here.
- The hook for the rest of the talk: GRAPE is told only where each start state must end up. What the state does in the middle of the pulse is whatever the optimiser found. Next slide shows what that looks like.
- The detailed loop (simulate, score, gradient, step) is in the pulse-optimisation part, on the "What GRAPE does" slide.

- Follow $|0_L\rangle$ through an X gate, which should turn it into $|1_L\rangle = |2\rangle$.
- Left: half the weight on 0 photons, half on 4. All inside the green stripes, which mark the Fock states the code uses.
- Middle: once the drives are on, weight shows up on odd photon numbers and above 4, and the transmon is partly excited. At this moment the state is not a codeword at all.
- Right: at the end the optimiser has brought everything back to 2 photons and the transmon to g. The gate is correct.
- So "the code space" is only a statement about the endpoints. In between, the parity check we are about to meet would not even give a sensible answer.
- Why it has to spread, in three steps. (1) Only the cavity drive $\epsilon\,\hat{a} + \text{h.c.}$ changes photon number; the transmon drive, χ and Kerr all keep n fixed. And it changes n one photon at a time.
- (2) So an X gate cannot avoid odd photon numbers: going from 4 to 2 passes through 3, and from 0 to 2 through 1. That is the purple.
- (3) A displacement moves every Fock component at once; it cannot tell 0 photons from 4. Only χ treats photon numbers differently. So the gate spreads the state out with the cavity drive, lets χ and the transmon drive act differently on each n, and brings it back. The spread is what gives χ something to act on.
- If asked "is it a coherent state?": displacing the vacuum gives one, but here the start is $|0\rangle + |4\rangle$ and χ acts throughout, so it is a displaced, transmon-entangled mix with a similar spread, about $|\alpha|$ either side.
- If asked why it reaches Fock $\sim$10: a 3 MHz drive for 100 ns displaces by $|\alpha| \approx 1.9$, about 3.5 extra photons on top of $|4\rangle$. GRAPE is only told the endpoints and the gate is short (about four periods of $1/|\chi|$), so it uses the room. That last part is our reading, not the paper's.
- These numbers are a cartoon. Say so, and move to the paper's simulation.

- This is the paper's own simulation of the two X gates the talk is about. Grey is the probability of each Fock state, on a log scale, against time.
- At $t = 0$ the dark rows are 0 and 4 ($|0_L\rangle$); at the end the dark row is 2 ($|1_L\rangle$). Exactly the cartoon.
- In between both gates fill Fock states up to about 10. Neither stays in the code space; that is normal for a fast optimised gate.
- The two pulses look alike to the eye. The difference between Ord and EsT is not *whether* they leave the code space but *how*: that is the whole subject of the theory part.
- The drive traces also show what GRAPE output looks like: a few MHz, wiggling on a ten-ns scale.

- Left: the codewords use only even photon numbers. Lose one photon and you land on odd photon numbers: $|0_L\rangle$ (0 and 4) goes to 3, $|1_L\rangle$ (2) goes to 1. The 0 component simply cannot lose a photon.
- So the question "did we lose a photon?" is the question "is the parity odd?".
- Right: the standard parity check, a Ramsey sequence on the transmon. Because of χ, the transmon precesses at a rate that depends on n. Wait $\pi/|\chi| = 1/(2 \times 3.66\ \text{MHz}) \approx 137$ ns and the phase is $n\pi$: the same for all even n, opposite for all odd n.
- The second $\pi/2$ turns that phase into g or e, and the readout gives the parity in one shot.
- The important part for QEC: the measurement does not tell you how many photons, nor whether you had $|0_L\rangle$ or $|1_L\rangle$. It only tells you whether an error happened.
- In the paper this check is used mainly to sort shots: even $=$ code space, odd $=$ error space.

- The whole error-correction cycle on one line. Start with any logical state $\alpha\,|0_L\rangle + \beta\,|1_L\rangle$.
- A photon leaks out at some random moment. The state becomes $\alpha\,|3\rangle + \beta\,|1\rangle$ — different photon numbers, but the same α and β.
- The parity check sees odd. A recovery pulse maps 3 back to the $|0_L\rangle$ combination and 1 back to 2. The logical state is restored.
- Why α and β survive: both codewords lose a photon with the same amplitude $\sqrt{2}$, because both hold 2 photons on average. If one held more, it would be more likely to be the one that lost a photon, and the loss would leak which-codeword information. Then nothing could undo it.
- Even outcome: no jump, but "no photon lost" is itself slightly informative — the 4-photon part decays faster than the 0-photon part. That small known distortion is also undone by the recovery. Do not dwell on it.
- Limit: one loss per check interval. Two losses land back in even parity and are missed.
- This is the idling story, and it works: Hu 2019 with exactly this code, and several groups past break-even since.
- This slide is the plain-language version of the Knill–Laflamme and "why the kitten code works" slides later on.

- Now put a gate between two correction rounds and ask where the loss can happen.
- After the gate (green): exactly the idling case of the last slide. Fine.
- During the gate (red): the gate is halfway through, the state is the spread-out mid-gate state from the cartoon, and the photon leaves from *that*. The rest of the pulse then acts on a state it was never designed for. Where it ends depends on when the photon left.
- The recovery is one fixed operation. It cannot undo a whole family of different endpoints, and we do not know t.
- Before the gate (grey): the gate then acts on an odd-parity error word. An ordinary gate was only told what to do with codewords, so this is also bad unless the gate behaves well on the error space too — which is the idea of the fix.
- The size of the problem: about 2 photons, a 1 μs gate, 180 μs lifetime: roughly 1% of gates. For a memory that is protected, that is a big hole.
- The formal version of this slide comes back as "Why a gate breaks all of that".

- The paper's own cartoon of the last slide. Green box: code space; purple box: error space; the curve is the gate; crosses are photon losses.
- (a) Ordinary gate: after a loss the state follows some other path in the error space, a different one for each loss time. The QEC arrows return to different places in the code space. Wrong answers.
- (b) Error-transparent gate: the error space runs the same gate in parallel. A loss at any time just moves you to the twin path, and you end at one known point. One recovery fixes all of them.
- Where the field is: memories past break-even, but transparent gates only for phase gates, where the state does not need to move between Fock states much.
- The paper's claim: X, H, T — a universal set — with the same linear drives as everyone else, transparent only approximately. Measured: about five-fold lower infidelity, conditioned on a photon loss during the gate. We do not reproduce the experiment.
- Bridge: the next three slides are the hardware they did it on.

- The usual Yale-style 3D package. The tall cylinder is a $\lambda/4$ coaxial cavity: the stub in the middle is the centre conductor, and the mode lives around it.
- The transmon chip slides in through a side tunnel; its pad pokes into the cavity to couple to it. The same chip carries the readout resonator.
- Fabrication details, if asked: Nb pads sputtered and patterned by photolithography, Al/AlO$_x$/Al Dolan-bridge junction, junction area 0.047 μm^2, chip clamped in with copper.
- Worth saying for an experimental audience: the cavity T_1 is low for this design, 180 μs. The paper blames surface degradation after shipping and shelf time. Their results are therefore not limited by an unusually good cavity.
- Same package as their earlier paper (Ref. [25] of the paper), with small wiring changes.

- Standard cQED wiring; walk it top to bottom, briefly.
- Room temperature: one OPX+ channel pair per drive makes baseband I and Q; the Octave up-converts with IQ mixers. Cavity and transmon share a local oscillator, readout up- and down-conversion share another. All pulses are triggered with a 100 ns buffer.
- Down the fridge: each input line is attenuated in stages (qubit 50 dB, cavity 53 dB of cold attenuation; readout 60 dB plus 20 dB at room temperature), low-pass filtered at 12 GHz, and HERD-filtered against infrared.
- Output: the readout signal goes through a circulator to a TWPA (itself pulsed only around the readout), then a HEMT at 4 K and a room-temperature amplifier.
- Nothing exotic — which is the paper's point: the gates need no extra hardware, only different waveforms on the same two drive lines.

- The paper's Table I, retyped. Left: frequencies and couplings, each with the term it puts in the Hamiltonian. Right: coherence and thermal populations.
- The paper's table writes the cavity term as $\omega_c a^\dagger a$; it is the same ω_a.
- The numbers to remember: $\chi/2\pi = -3.66$ MHz (fast, selective control), cavity $T_1 = 180$ μs, transmon $T_1 = 70$ μs, and a gate of about 1 μs.
- The cavity Ramsey time, 290 μs, is below $2\,T_1 = 360$ μs, so there is some cavity dephasing too, partly from transmon thermal population through χ. Photon loss is still the main error.
- Transmon dephasing ($T_{2,R} = 30$ μs) is poor, partly because of the two-level system noted in the caption. The paper post-selects on the transmon ending in g to focus on photon loss.
- The Kerr and χ' rows look negligible. Ask the audience to remember them: they are exactly what will stop the gate from being perfectly transparent.
- Next slide: the same table written as the Hamiltonian the theory uses.

- Two modes. The cavity $\hat{a}$ is where the logical qubit lives; the transmon $\hat{q}$ is a nonlinear ancilla we use to steer the cavity and to read out parity.
- H_a: the cavity's bare frequency, plus self-Kerr K and a smaller second-order self-Kerr K'. Both make the photon energy depend on how many photons are already there.
- H_q: transmon frequency plus anharmonicity K_q. At -180 MHz this is what makes the transmon usable as a qubit at all — and it is why the simulation keeps three transmon levels, not two.
- H_{int}: the dispersive coupling χ, plus a second-order correction χ'. Both are photon-number dependent shifts conditioned on the transmon.
- H_{drive}: the only thing we control. Two complex amplitudes, linear in the mode operators.
- These are exactly the numbers and operator forms in `EST/device.py`.
- If asked why the main text shows fewer terms: Eq. 9 is the low-order version; App. A adds K' and χ', and χ' matters later — it is one of the two terms that make exact transparency impossible.

- The first of two rules that any code must pass to correct a photon loss. Say it with the pictures, no equations.
- Left, the kitten code. A loss moves 0_L (0 and 4 photons) to 3 photons, and 1_L (2 photons) to 1 photon. Two different places, both odd. The codewords are still distinct after the loss, and the parity check can tell an error happened.
- Right, the simplest idea one might try: store the qubit as 0 or 1 photon. Lose the photon and 1_L becomes 0_L. Nothing can tell "an error happened" from "it was 0_L". The information is gone.
- So rule 1: a loss must not map one codeword onto another, or onto anything that overlaps the code. In the formal language this is the "no mixing" half of Knill–Laflamme (appendix).

- The second rule is subtler, and it is where the kitten code's numbers come from.
- Think of the cavity as a leaky bucket: photons drip out at a rate proportional to how many are inside. Each drip is one photon loss.
- Kitten code: 1_L holds exactly 2 photons; 0_L holds 0 or 4, so 2 on average. They drip at the same rate. Seeing a drip tells you nothing about which codeword you had.
- Bad code, say 2 photons and 4 photons. It passes rule 1 (the losses land on 1 and 3, different places), but the 4-photon codeword drips twice as fast. Seeing a drip makes 1_L more likely: the environment has partly measured the qubit.
- Right: draw the logical state as an arrow, here an equal superposition on the equator. Kitten: after the loss the arrow is where it was, just stored in 3 and 1 photons. Bad code: the arrow is tilted toward the leakier codeword. Recovery cannot untilt it, because it does not know the original arrow.
- Fine print if asked: "no drip" is a clue too — the 4-photon part of 0_L decays faster than the 0-photon part. That small, known distortion is fixed by the recovery as well; it is the same rule at first order.

- Put the two rules together in one picture. Draw the logical qubit as a sphere (here a circle) with an arrow for the state.
- Left: what a correctable loss does. The whole sphere is copied into the error space (odd photon numbers): same size, same arrow, nothing lost. Only the label changes, from (0 and 4, 2) to (3, 1).
- Middle: rule 1 broken. The copy lands on top of the code, so "error" and "no error" cannot be told apart.
- Right: rule 2 broken. The copy is squashed, so the arrow is tilted. That is the partial measurement from the bucket slide.
- This is the content of the Knill–Laflamme theorem (Knill and Laflamme, 1997): those two conditions are necessary and sufficient. Necessary: break either one and no recovery works. Sufficient: if both hold, a recovery exists (measure which copy, rotate it back). It says *whether* a code can correct a set of errors, not how to build the recovery.
- Read the equation: sandwich any product of two errors between two codewords. The answer must vanish unless the codewords are the same (δ_{ij}, rule 1), and must not depend on which codeword it is (α_{ab}, rule 2).
- For the kitten code with errors $\{\mathbb{I}, \hat{a}\}$: $\mathbb{I}^\dagger\mathbb{I}$ gives 1, $\hat{a}^\dagger\hat{a}$ gives $\bar{n} = 2$ on both codewords, and the cross term $\mathbb{I}^\dagger\hat{a}$ vanishes because $\hat{a}$ flips parity. So $\alpha = \mathrm{diag}(1, 2)$: it passes. The proof is in the appendix.
- For the rest of the talk, "correctable" means exactly this picture.

- This is the picture of slide 10 (the paper's Fig. 1a–b) with one addition: the arrow inside each circle is the logical state, so we can see what "the same" means.
- Top band of each panel: the code space. During an X gate the arrow turns from 0_L (up) to 1_L (down).
- A photon leaves at some moment (the star). By the two rules, the arrow drops onto the error track pointing the same way — nothing is lost yet.
- Left, ordinary gate: the drives were only designed for the code track. On the error track they turn the arrow some other way, so it ends somewhere wrong, and somewhere different for each loss time.
- Right, error-transparent gate: the drives turn the arrow on the error track in step with the code track. Whenever the photon leaves, the arrow ends as "X done, plus one photon lost": the same known state. One parity check and one recovery fix it.
- The one-sentence definition: a gate is transparent to photon loss if losing a photon during the gate is the same as losing it after the gate.
- Next slide: the same statement as one equation, for those who want it.

- State the equation in words first: the two routes around the square agree, up to an overall phase that may depend on t.
- The phase $e^{i\phi_j(t)}$ is free because a global phase is physically invisible — it does not appear in ρ.
- What this buys: collapse a continuum of possible outcomes (one per jump time) into a single outcome. The error becomes *transparent* to the gate — it commutes through, up to a phase.
- Concretely: after the gate, you measure parity, find you are in the error space, and apply the same recovery you would have used for an idling qubit. The gate did not scramble the error.
- Heads-up for later: this exact square, with the subspaces allowed to move, is what the three conditions in a few slides are designed to enforce.
- If asked about multiple losses: the framework is built for single loss, the dominant channel for this code. Two-photon events are not covered.

- Ask term by term: if a photon leaves in the middle, does this part of the Hamiltonian treat the state after the loss the same way it would have? If yes, the loss "passes through" that term.
- The cavity drive: a push adds or removes amplitude regardless of how many photons are present. Losing a photon before or after a push gives the same thing. Passes.
- χ: the transmon shifts by 3.66 MHz per photon. After a loss every component has one photon fewer, so everything shifts by exactly one notch. That is a known, state-independent change, which the recovery can account for. Passes.
- Kerr ($K/2\pi = -22$ kHz) and the second-order χ': each photon's energy depends on how many photons are already there — the ladder's gaps shrink as you go up. Then the loss from 4 to 3 and the loss from 2 to 1 shift the state's frequency ($E_n/\hbar$, how fast its phase winds) by different amounts, $3K$ and K. That difference is between the two codewords, so it is logical. Fails.
- The drives are what we control; Kerr and χ' are built into the hardware. The next slide says why that one failing term is enough to rule out exact transparency.

- Why one small term matters. Each Fock state's phase winds at its own frequency, $E_n/\hbar$. The logical arrow is set by the frequency difference between the two codewords.
- With Kerr ($E_n = \frac{K}{2} n(n-1)$), the pair (4, 2) differs in frequency by $5K$ before a loss and the pair (3, 1) by $3K$ after it. So after the loss the arrow slowly turns, and the extra turn is proportional to how long ago the photon left.
- If the loss time were known, that turn could be undone. It is random. So different shots end with different turns — the fan in the picture — and averaged over shots the arrow shrinks. That is logical dephasing, and no recovery can undo it.
- Size: the change in frequency difference is $2K$, about $2\pi \times 44$ kHz, so about 0.3 rad per microsecond at the code's own photon numbers. Mid-gate the state reaches Fock 10 (slide 6), where the Kerr differences are bigger.
- Why the drives cannot fix it: to cancel an effect that grows with photon number like Kerr, you need a control that grows the same way. A linear drive ignores photon number; χ is the same per photon. Nothing in the toolkit matches it.
- So exact transparency is "not expected" (the paper's words). What GRAPE can do is arrange the path so the damage stays small. That is what "semi" means.

- The trick that makes the paper's analysis work: stop insisting that mid-gate the state be in the original code. Of course it is not — slide 5 showed it spreading out.
- Instead, follow the codewords. Run the ideal gate (no loss) on 0_L and on 1_L separately. At any moment, each has become some spread-out state; the bars show cartoons of those.
- Those two states, at that moment, are the code "right now": the instantaneous code space. Any logical state is the same superposition of them, so the arrow rides along.
- At the end of an X gate, the old 0_L has become 2 photons and the old 1_L has become 0 and 4: the code space is back where it started, with the labels swapped.
- The green ribbon is the cartoon of that moving 2-dimensional space through the much bigger space of the cavity and transmon.
- Bars are made up; the real ones are like slide 6.

- Same idea for the error space. At any moment, ask: if a photon left right now, where would the current codewords land? Those two states are the error space right now.
- At $t = 0$ and $t = T$ that is the familiar odd-parity pair, 1 and 3 photons. Mid-gate it is something spread out, like the code itself (bars are cartoons).
- The vertical arrows are photon losses at different moments; each lands on the purple ribbon at that moment, with the arrow pointing the same way (if the two rules hold right then).
- Now error transparency has a clean picture: after landing on the purple ribbon, the rest of the gate should keep the state on the purple ribbon and turn its arrow exactly as it turns the green one. Then every loss, at any time, ends at the same place.
- That splits into three things to check, on the next slide.

- The paper turns “transparent” into three checks, and gives each a number that is zero when the check passes.
- Check 1: at this moment, would a loss be correctable? That is the two rules, applied to the code as it is right now. The number is called the QEC violation.
- Check 2: once a photon is lost, does the rest of the gate keep the state on the moving error ribbon, or does it drift off into states the recovery does not expect? The number is the leakage.
- Check 3: on the two ribbons, do the arrows turn together? The number is the trajectory mismatch: how far apart the code arrow and the error arrow point.
- All three zero at every moment is exact transparency, which linear drives cannot reach. The EsT gate is the one whose pulse was optimised to keep them small.
- These are the curves on the paper’s Fig. 1d–f and on our comparison slides near the end. Formulas for all three, and the conditions behind them, are in the appendix.

- This is the measurement behind all the results that follow; worth a minute.
- First box: a transmon rotation (identity, $\pm X/2$, $Y/2$, X) picks one of the test states; the encode pulse maps it from the transmon into the kitten code in the cavity.
- Then N copies of the gate, EsT or Ord.
- Error detection: read the transmon first and keep only shots where it ended in g, so transmon errors do not muddy the picture — the gates are only meant to be transparent to cavity loss. Then the parity check sorts each shot: even means no loss, odd means a photon left at some point during the N gates.
- Decode: separate pulses map the even (code) or odd (error) cavity state back onto the transmon, and transmon tomography gives a process fidelity for each sort.
- The number to watch is the error-space fidelity: given that a photon was lost, how much of the logical information made it. 0.25 is the value for a completely scrambled qubit, which is the floor on the plots.
- Note there is no correction here: the parity check only sorts. Correction comes with AQEC later.

- Left, the measured process fidelity against the number of X gates. Circles: shots sorted into the code space. Triangles: shots sorted into the error space. Blue EsT, red Ord. Lines are the paper's model.
- Code space: blue and red lie on top of each other. The transparency cost did not make the gate worse when nothing goes wrong. That was not guaranteed.
- Error space: this is the point of the paper. When a photon was lost, the EsT gate keeps much more of the logical state: about 0.5 against 0.3 at ten gates.
- Why the error-space curve first rises: after only a few gates, a photon has rarely been lost, so most shots sorted as odd are actually mistakes — a little error-space weight from state preparation, and the parity check itself is wrong about 5% of the time. Those shots carry no logical information. As N grows, real losses take over.
- Right: the model with parity-readout errors removed, the "intrinsic" behaviour. Here the gap between EsT and Ord in the error space is clean from the first gate.

- The same experiment for the other two gates of the set. Top row Hadamard, bottom row T.
- Left column: as for X, code space the same for EsT and Ord, error space clearly higher for EsT.
- Middle: the model with the parity-readout errors taken out, as on the X slide.
- Right: two populations against gate number. Squares: probability the transmon is still in g at the end (transmon decay pulls it down). Crosses: probability the parity is odd, i.e. a photon was lost; it grows with N.
- For H the EsT gate ends odd slightly more often than Ord. The paper puts this down to a higher average photon number during the EsT pulse — more photons, more losses. Transparency is about what happens *after* a loss, not about having fewer.
- For T, the paper notes a small mismatch between model and data for the Ord gate's error space that it does not fully understand.

- The headline number, and where it comes from. Each decay curve is fitted with the paper's model, which separates out the parity-readout errors and gives a per-gate fidelity.
- Left half: the case transparency is about. A photon is lost during one gate, so that gate starts in the code space and ends in the error space. Ord loses about half the logical information in that case; EsT about 15% for X. Averaged over X, H, T, the infidelity is about five times lower. This is the "five-fold" of the abstract.
- Right half: a gate applied to a state that is already in the error space (a loss happened earlier). The EsT pulses were also optimised to work there, and they are 15–21 times better. The Ord pulses were never asked to.
- Be careful with the claim: these are per-gate numbers from a model fit to the data, not a direct single-gate measurement.

- Now the question that matters: does transparency still pay off once you actually correct the error, rather than just sort shots?
- Sequence (a): N gates, then a single autonomous correction step (AQEC). The AQEC pulse is itself GRAPE-optimised, 1 μs long; it maps the odd-parity error states back into the code and moves the entropy into the transmon. The transmon is then reset, which takes another 1.2 μs.
- Plot (b): fidelity after the whole sequence, against N. Grey: idling with no correction. Orange: idling with AQEC, the best case. Blue and red: EsT and Ord gates followed by AQEC.
- EsT beats Ord, and the gap grows with N at first — more gates means more chance a photon was lost mid-gate, which is exactly the case EsT handles. The gap peaks around 0.17 near 10 gates (Fig. 4c of the paper, not shown).
- Both gate curves still fall faster than idling with AQEC: errors the scheme does not cover (two photons lost, transmon decay or dephasing during the gate, control errors) accumulate.

- Say clearly that this one is a simulation, not data. It uses the same pulses and the measured coherence times, but no control errors.
- Left (a, b): the lab sequence, one AQEC step after N gates. Right (c, d): an AQEC step after every gate, which is what you would do in practice.
- The third curve, LE, is a control experiment the paper designed: a pulse optimised to work well in both the code space and the error space, but without the transparency cost. It lands between Ord and EsT.
- So the ordering EsT $>$ LE $>$ Ord says the gain is not just "the pulse also works in the error space": making a loss *mid-gate* harmless (transparency) adds on top.
- With correction after every gate, the EsT advantage is larger: 0.72 against 0.61 after 10 gates.
- The paper presents this as evidence that the EsT gain holds without control errors, i.e. it comes from the pulses themselves.

- GRAPE is Gradient Ascent Pulse Engineering (Khaneja et al., 2005). The name says "ascent" because it was written to maximise a fidelity; minimising a cost is the same thing with the sign flipped.
- It is *open loop and model based*: the pulse is designed entirely on a computer model of the device, then sent to the hardware as is. Nothing is learned from the experiment during the optimisation.
- This opens the second half. The theory told us what a good gate looks like; now we need a way to actually find a pulse that does it.
- The representation first. The control hardware plays one value per nanosecond, so the pulse is a staircase of N steps. On each step there are four real numbers: the I and Q amplitudes of the cavity drive and of the transmon drive (the I/Q slide explains them). Stacked end to end that is $4N$ numbers — 4000 for a 1 μs gate. To the optimiser, "the pulse" is just this vector u.
- Then walk the loop. Simulate, score, take the gradient, step. The trick that makes it practical: the gradient with respect to *all* the numbers costs only about one extra simulation, not 4000.
- The paper's tools (App. C): Dynamiqs to integrate the Schrödinger equation, JAX for automatic differentiation, and the Adam optimiser from Optax.
- Ours, if asked: JAX autodiff for the gradient, but scipy's L-BFGS-B as the optimiser, plus a second, pure-numpy pipeline with a hand-derived gradient that agrees with it. Same cost, different step rule.
- The next four slides fill in the loop: the model that is simulated, the controls as real numbers, how a staircase pulse becomes a gate and a gradient, and which states go in. Then the cost.

- "Characterising the system for optimisation" means: decide exactly what the computer simulates. Three decisions — the frame, the truncation, and what to leave out.
- The frame. App. C says both drives are applied on resonance. In the frame rotating at ω_a and ω_q the two big frequencies vanish. Every other term commutes with both photon-number operators, so it survives unchanged. That leftover is the drift H_0: always on, not under our control.
- The truncation is *ours*; the paper does not state one. Twenty cavity levels is generous — the code words stop at $|4\rangle$ and the trained pulses reach about Fock 9 or 10. We checked it on held-out truncations from 16 to 28: the fidelity moves by less than 10^{-5}.
- Three transmon levels, not two, because the anharmonicity K_q is finite: a strong drive on $g \to e$ can also push population to f, and a two-level model would hide that.
- The K' term $\frac{K'}{6}\,\hat{a}^{\dagger 3}\hat{a}^3$ is exactly as Table I writes it. It is a 590 Hz term and barely matters.
- The key point to repeat: the optimisation is lossless. No jump is ever simulated. The annihilation operator appears only as an algebraic object inside C_2, just as it did in the metrics. Decoherence is assessed afterwards, not trained against.

- The Hamiltonian on the device slide has complex drives $\epsilon(t)$ and $\Omega(t)$. An optimiser wants a plain vector of real numbers, so split each into a real part I (in-phase) and an imaginary part Q (quadrature).
- Check the identity in one line: $(\epsilon_I + i\epsilon_Q)\hat{a} + (\epsilon_I - i\epsilon_Q)\hat{a}^\dagger = \epsilon_I(\hat{a} + \hat{a}^\dagger) + i\epsilon_Q(\hat{a} - \hat{a}^\dagger)$. Both operators on the right are Hermitian — they are, up to $\sqrt{2}$, the position-like and momentum-like quadratures of the mode.
- What this buys. One: four real control functions instead of two complex ones. Two: four *fixed* Hermitian operators H_j, with the pulse entering only as real coefficients, so H is linear in the controls. That makes the derivative with respect to each control trivial: $\partial H/\partial u_j = H_j$.
- Three: it is how the hardware works. App. B: the pulses come out of an AWG and go through IQ mixers. The two numbers per drive per nanosecond are literally the two voltages the AWG plays.
- So a pulse is an $N \times 4$ real array: 4000 numbers for X and H. This is the same column order as our saved files in `pulses/est/`.
- Point at the picture: the drive is a vector in the I–Q plane. The amplitude limit is a *circle* in that plane. Hold that thought for the constraints slide.
- If asked about the sign of Q: it is a convention. Our code writes the drive as $\epsilon\,\hat{a}^\dagger + \mathrm{h.c.}$ instead of $\epsilon\,\hat{a} + \mathrm{h.c.}$, so there $\epsilon = u_0 - iu_1$. Same operators, same pulse array; only the sign of Q flips, and the amplitude cap depends only on $|\epsilon|$.

- This is the mechanics inside the GRAPE loop's "simulate" and "gradient" boxes: how a pulse turns into a gate, and why its gradient is cheap.
- Within one nanosecond the controls are constant, so the Hamiltonian is constant and the evolution over that step is a single matrix exponential. The whole gate is the product of N of them, latest on the left.
- The states the cost needs — the six code states, their six error partners, at every step — are all obtained by applying these U_k one after another.
- The gradient. The cost depends on $u_j(k)$ only through U_k, so the chain rule needs the states just before step k and a "backward" state coming from the cost side. One forward sweep and one backward sweep give the derivative for all 4000 numbers. In cost, that is about two to three simulations, not 4000.
- The paper lets JAX do this automatically. So do we, with one care point: we take the exponential with `expm`, not by diagonalising H_k, because differentiating an eigendecomposition blows up when eigenvalues are degenerate.
- Then the optimiser takes the step: Adam in the paper, L-BFGS-B in our code.

- The optimiser does not work with the full 60×60 propagator. It evolves a handful of states and compares them with targets. App. C says so explicitly: simulating states is cheaper than simulating the propagator.
- Why six and not just $|0_L\rangle$ and $|1_L\rangle$: two basis states fix where each one goes but not the relative phase between them. The six cardinal points pin that phase down; together they are what is called a 2-design. Only the physically meaningless global phase is left free.
- The error partners are the same six states pushed through $\hat{a}$ and normalised. Because $\hat{a}$ maps the kitten code onto the error space without distortion (the equal-$\sqrt{2}$ fact), the partner of $|+X_L\rangle$ really is the error-space $|+X\rangle$. They are used by C_2.
- Gate lengths are the paper's: X and H use both drives for 1 μs; T is a phase gate and uses the transmon only, for 0.6 μs.
- One nanosecond is the time resolution of the control hardware, so the simulation uses the same grid the AWG will play.
- Fairness note from the main text: EsT and Ord pulses are compared at the same maximum active Fock level, the highest n that reaches population above 0.01.

- One number for the optimiser to push down: a weighted sum of separate costs. The weights are tuned by hand (App. C says "manually tuned").
- Read down the table. C_1 is the only one that looks at the end of the pulse. C_2 and C_3 are *running* costs: they look at the whole trajectory. That is the structural difference from ordinary gate optimisation — a loss can strike at any time, so the middle of the pulse has to be scored too.
- C_4 is ours. The paper states a 4 MHz amplitude limit but not how it was imposed. We impose it as a penalty; the constraints slide says why.
- The block is the caveat that used to come later: the three metrics we just defined are *not* what is optimised. The paper's own words (p. 4): "to facilitate differentiability, we utilize a cost function based on state overlaps and fidelities rather than commutator norms and projectors." The metrics are then used to check the result.
- App. C lists optional extra costs: (a) the mean-photon-number difference between the evolving code words (Eq. A7, aimed at Condition 1), (b) smoothness, (c) robustness. None is in the default recipe.
- The last sentence of the block heads off a natural misreading: three conditions, three metrics, three costs does *not* mean one cost per condition. C_2 is one overlap that pushes on tracking and matched dynamics together; Condition 1 is only touched indirectly; C_1 is the gate and C_3 is scaffolding.
- If asked whether we tried (a): yes. It more than halves Δ_{QEC}, but makes end-of-pulse leakage and $\eta_{\mathcal{E}_j,\psi}$ worse. The three conditions trade against each other.

- The standard gate cost. Each term is the overlap between where a cardinal state actually ends up and where it should end up; $|\cdot|^2$ makes it a probability. Average over six, subtract from one.
- Because all six cardinal states are in the average, a pulse cannot score well by getting $|0_L\rangle$ and $|1_L\rangle$ right with the wrong relative phase.
- This is the only cost an Ord gate uses. It says nothing about the middle of the pulse, so an Ord pulse is free to wander anywhere mid-gate — exactly the behaviour the "why a gate breaks all of that" slide warned about.
- The trade in one number, from our own X pulses: Ord reaches $F_{\mathrm{target}} \approx 0.99999$; EsT stops at about 0.9993, because part of its budget goes into transparency.

- This is the cost that makes a gate error-transparent, and it is the commuting square written as one overlap per time step.
- Two routes, as on the commuting-square slide. Route 1: the photon is lost at $t = 0$ and the error state is carried forward by the drive — that is $\psi_{E_j}(t)$, evolved on its own. Route 2: no loss until now, then the loss — that is $\hat{a}\psi_C(t)$. Transparency says the two routes give the same state.
- $\mathcal{N}$ is there because $\hat{a}$ is not unitary: $\hat{a}\,|\psi_C\rangle$ has length $\sqrt{\bar{n}}$, not 1, so the overlap has to be normalised.
- The paper's claim: this single overlap "partially" enforces all three conditions. It most directly enforces tracking and matched dynamics (2 and 3): a mismatch in either subspace or position inside it lowers the overlap. Condition 1, and so Δ_{QEC}, is only weakly targeted — App. A says as much, and it is why Δ_{QEC} improves least.
- The footnote, our deviation: the paper prints a single power of $\mathcal{N}$. With that, F_{ET} is not bounded by 1 — it can reach about $\sqrt{\bar{n}}$ — so the cost rewards simply adding photons. Dividing by $\mathcal{N}^2$ makes it a true fidelity between 0 and 1. We kept the printed form as an option and tested it: it is not more transparent.
- The sum stops at $N - 1$, so the final time is not included; the endpoint is covered by C_1.

- The loophole is real and the optimiser finds it. C_2 is an average over every step. If the gate happens in a short burst and the state sits still for the rest of the pulse, most steps are trivially transparent and score well, while the few steps that do the gate — where a photon loss would actually hurt — barely count. App. C: the regulariser ensures "the optimizer does not simply minimize the pulse dynamics duration to artificially inflate F_{ET}".
- Why "at the end": a burst early on would leave the code and error states mismatched for the rest of the pulse, and every later step would score badly. Only a late burst keeps the mismatch confined to a few steps. "Nearly" because Kerr still acts on an idle state.
- Consistent with this, not proof of it: in our T-gate runs, the stage with C_3 switched off put about a third of the transmon drive energy into the last 48 ns.
- The paper's fix: look at how fast the code state moves at each step, and penalise the variance of that speed over the pulse. The distance used is the Fubini–Study distance, the natural distance between two quantum states.
- Why a variance is the right cure: a burst followed by stillness has a very uneven speed — fast for a moment, then zero — so its variance is large. A gate spread over the whole duration moves at a steady pace and pays little.
- The paper is explicit (App. C) that this term "does not improve ET performance directly". It is scaffolding. It also makes EsT and Ord gates use their time in a comparable way, so incoherent errors affect both alike.
- The footnote, our deviation. The printed C_3 has units of frequency squared, so its size depends on the time unit: about 965 in rad per microsecond, about 10^{-3} in rad per nanosecond, for the same pulse. It is also smallest when the drives are switched off — it rewards slowing the whole pulse down, which is not what it is for. Dividing by the mean speed squared removes the units and the bias towards slowing down.
- Honest statement if pressed: C_3 does not stop a pulse that barely drives at all. C_1 does, because such a pulse does not do the gate.

- Why two stages: asking for high fidelity and high transparency at once from a random start tends to get neither. So the paper first builds transparency with a strong C_2 and C_3, then turns C_3 off and C_2 down to finish the gate.
- Stage 1 numbers from App. C: transparency fidelity up to 0.83, gate fidelity only 0.9–0.99 — not yet a usable gate. Stage 2 lifts the gate fidelity while the transparency moves by about a percent.
- Our weights, if asked: $(w_1, w_2, w_3, w_4) = (1, 0.7, 7, 1)$ then $(1, 0.1, 0, 1)$, inside the paper's ranges, with C_4 on in both stages.
- The block is the experimental design. Ord runs the same code with only C_1. That is the comparison every EsT result is quoted against, so it matters that the two differ in the cost and nothing else.
- The caveat in the block: each run starts from a random pulse and lands in a local optimum, so one EsT–Ord pair could differ by luck. Our comparisons are mostly single-seed so far. Where we checked two seeds, the seed-to-seed change in end-of-gate leakage was about 0.02, far below the EsT–Ord gap in mean leakage (about 0.26 vs 0.74 on X). Say that, and say it is two seeds.
- We also found that a single stage — stage-1 weights all the way — does not finish the X gate: holding the velocity term on to the end dominates the cost. That is good evidence that the second stage is doing real work.

- Keep this brief. These are the rules that make a pulse playable on real hardware and keep the simulation honest.
- Bandwidth: the AWG cannot produce frequencies above about 50 MHz, so the paper Fourier transforms the pulse and removes everything outside that band. App. C: it keeps the pulses reliable and the simulation close to the experiment.
- Amplitude: at most 4 MHz on each drive, the same for cavity and transmon.
- Ramp: a 48 ns Gaussian rise and fall, so the pulse switches on from zero and off to zero smoothly, instead of jumping.
- The 1 ns step and the transmon-only T gate we have already seen.
- How, in our code. The optimiser works on a raw vector x; we band-limit it, then multiply by the ramp envelope, then zero any drive that is switched off, and only the result u is physical. The mask and the ramp hold exactly. The band limit holds only to about 0.3% of the energy, because the ramp is applied after it. That order was chosen by measurement: the other order is exactly band-limited but smears the ramp so the ends reach a quarter of full scale.
- The amplitude cap cannot be done that way. The optimiser's bounds are a box, one interval per number, but $|\epsilon| \leq \epsilon_{\max}$ is a circle in the I–Q plane. So it is a penalty, C_4, that switches on only above the cap. Being soft, it lets the pulse sit about 0.2% over the cap on under 1% of the steps.

- Same three metrics, same gate, same duration, same linear axes. Top row is the paper's Fig. 1d–f, cut from the PDF; bottom row is our own Figure 2a from `est_experiments.ipynb`, recomputed from our trained pulses.
- Read it column by column. Left, QEC violation: both start and end near zero, because both pulses bring the code words back into the code at $t = T$; Ord peaks mid-pulse in both, EsT stays lower through the middle.
- Middle, leakage: Ord climbs towards 0.8–1 in both. The EsT curves are at a similar level mid-pulse. The visible difference is the very end: the paper's EsT drops to zero at $t = T$, ours stays near 0.4.
- Right, trajectory mismatch for $|0_L\rangle$: Ord reaches the maximum of 2 in both, EsT stays well below. At the very end the paper's Ord falls back to about 0.2; ours stays near 1.5.
- Our legend says "Bloch-vector mismatch" and the paper says "trajectory mismatch"; it is the same Eq. 8.
- The next slide puts numbers on this.

- Where the numbers come from. Paper time averages: every curve of Fig. 1d–f digitised from the PDF (`slides/digitise_paper_fig1.py`; 400 and 600 dpi agree to 0.004, and it reproduces the notebook's leakage 0.22 / 0.61). Paper peaks and $\eta(T)$: read off the figure by eye, good to about 0.05–0.1. Ours: computed from the trained pulses with the eigh propagator.
- The good news first: the top three rows, each metric averaged over the gate. QEC violation 0.16 / 0.48 against their 0.18 / 0.65, leakage 0.26 / 0.74 against 0.22 / 0.61, trajectory mismatch 0.17 / 1.11 against 0.17 / 0.99. Every pair is within about 25%, and EsT sits well below Ord in both. Same metrics, same gate, same ballpark: the replication is in the right place on average.
- Peaks: our Ord QEC violation tops out at 1.5 against their 1.9; EsT is about the same height, 0.68 vs 0.6, but ours peaks late, at 0.78 μs, where theirs peaks at 0.40 μs.
- The real difference is the endpoint, in bold. Their EsT gate finishes with leakage 0.013, essentially transparent. Ours finishes at 0.396. Their Ord pulse also returns to a small trajectory mismatch at $t = T$ (about 0.2); ours ends at 1.55.
- Why the endpoint matters: QEC runs at $t = T$. A gate that is transparent on average but leaks at the end hands the recovery an error state in the wrong place.

- The question from the last slide: why does our EsT gate still leak at the end? The paper's App. C says its EsT gate "optimizes for fidelity of operation in both code and error space", but the printed cost (Eqs. C1–C8) has no error-space term. So we added one: C_{err}, the gate fidelity on the error words at $t = T$, with weight 0.3, in stage 1 only (`est_optimization.ipynb`, section 7).
- Why seed 6. The first try, at seed 0, fixed the end leak but had one sharp Δ_{QEC} spike, 3.3 at 0.65 μs. So we trained the same pair at 13 seeds with a pick rule fixed before training: closed ($F_1 \geq 0.999$), peak $\Delta_{\mathrm{QEC}} \leq 0.6$, and at least as transparent as its own control. Only seed 6 passed. That is the green gate, final (both stages).
- Blue and red are exactly the pulses of the time-traces slide. Same scorer for all three: eigh propagator at $n_c = 20$; green matches the notebook's Figure 7c to 10^{-10}.
- Middle panel: the green leakage ends at 0.09, against 0.40 for our EsT. Mid-gate it is not lower everywhere; the term mainly removes the leak at the end of the gate.
- Left panel: no spike. Green's Δ_{QEC} stays below 0.4 the whole gate.
- Right panel, the price: green's η for $|0_L\rangle$ has a short peak near 1.0 at 0.64 μs, higher than our EsT's. It ends at the same place (0.10 vs 0.09).
- Against its own matched control in the notebook (an EsT gate retrained at seed 6, not the blue pulse here): $L(T)$ 0.09 vs 0.49, mean Δ_{QEC} 0.12 vs 0.25, F_{ET} 0.80 vs 0.72.
- What we already ruled out: our C_2 and C_3 differ from the printed equations, so we retrained X with the printed forms at the same weights, at two seeds. $L(T)$ came out 0.40 and 0.39, no change (`est_experiments.ipynb`, Appendix A).
- So the leak is a property of the objective, not of the device. Our hypothesis is that the paper's code has an error-space term its printed cost does not show. We cannot check that without their code.
- Caveats: one gate out of 13 seeds, and three seeds never drove (parked); the paper compares pulses matched in maximum active Fock level, ours are 9, 8 and 9; and the paper's numbers come from digitising or reading a figure.

- Keep this to what we have shown. Everything here is simulation; we have not run hardware.
- Two gradient codes: a JAX autodiff path and a numpy path with a hand-derived gradient. They are checked against each other and against finite differences.
- The two cost changes are the $\mathcal{N}^2$ in C_2 and the unit-free C_3, both explained on their slides.
- The 3–7 times is time-averaged leakage and Bloch mismatch, EsT against Ord. X: leakage 2.9 times, mismatch 6.6 times on $|0_L\rangle$. H: 4.5 and 5.2 times. Single seed for these headline pairs.
- Extra costs: the mean-photon-number penalty halves Δ_{QEC} but hurts leakage; an end-of-gate error-space fidelity term cuts $L(T)$ but can stall some seeds; a smoothness term raised the mismatch. The metrics trade against each other.
- Open: the end-of-gate leakage of about 0.4, against the paper's 0.013. A terminal error-space term cuts it to 0.09 on the gate picked from 13 seeds (seed 6), still well above the paper; whether the paper did something like this is the question. And we need more seeds before quoting any EsT–Ord ratio as a result.

- Two objects, then one warning.
- The projector: P is Hermitian and idempotent — the linear-algebra projection onto the code subspace. $U^\dagger U = \mathbb{I}$ on the logical space, but $P = UU^\dagger$ is *not* the identity on the full Hilbert space. That asymmetry is the whole point of encoding.
- The Kraus operators E_j are what we casually call "the errors". Same maths as simulating decoherence.
- The warning is load-bearing, not a footnote: $\{E_j\}$ is only defined up to a unitary recombination W. The proof of Knill–Laflamme *uses* this to pick a basis in which the error matrix is diagonal.
- If asked: this is why "correcting three errors" and "correcting a three-dimensional span of errors" are the same statement.

- Read the equation aloud in words before unpacking it: sandwich any product of two errors between the code projectors, and you must get back the projector itself times a number.
- "Proportional to the logical identity" is the phrase to repeat — it recurs verbatim as Condition 1 later in the talk.
- The codeword form makes the two demands visible. Spend time here.
- δ_{ij}: no leakage *between* codewords. If $E_a^\dagger E_b$ had an off-diagonal piece, an error would rotate $|0_L\rangle$ toward $|1_L\rangle$ and no measurement could undo it.
- α_{ab} not depending on i: no which-path information. This is the subtler half and the one the kitten code buys us on the next slides.
- Hermiticity of α is automatic — it comes out of the proof as a Gram matrix, hence also positive semidefinite.
- Do NOT go through the full proof. The next slide sketches why the condition is enough; the derivation is in backup.

- This replaces three derivation slides; they are in backup if someone asks.
- Step 1 uses the freedom from the setup slide: any unitary recombination of the Kraus operators is the same channel. Pick the one that diagonalises α.
- Step 2: after that, F_k restricted to the code is $\sqrt{d_k}$ times a rigid move onto a copy $\mathcal{C}_k$, and different k give orthogonal copies.
- Step 3: orthogonal copies can be told apart by a projective measurement; rigid copies can be rotated back. The outcome probability d_k is the same for every code state, so the measurement reveals nothing logical. That is the "α independent of i" half of Knill–Laflamme, seen as a measurement property.
- Kitten preview: the copies are the even and odd parity sectors, so the syndrome is a parity measurement.

- Use the non-uniqueness from two slides ago. That freedom is the whole trick.
- Step 1: α is Hermitian (in fact a Gram matrix, so positive semidefinite), so it has a unitary eigenbasis and real, non-negative eigenvalues d_k.
- Step 2: rotate the Kraus set by that same unitary. Same channel, new labels.
- Step 3: substitute. $PF_k^\dagger F_l P = \sum_{a,b} \bar{u}_{ak}\, \alpha_{ab}\, u_{bl}\, P$, and that sum is just $(u^\dagger \alpha u)_{kl} = d_k \delta_{kl}$. Only the diagonal is left, hence the δ_{kl}.
- What the δ_{kl} buys: the original E_a could overlap on the code, so you could not cleanly tell them apart. The new F_k cannot.
- What the d_k means: $\| F_k \, |\psi\rangle \, \|^2 = d_k$ for every code state, so learning "k happened" reveals nothing about the logical state.
- If some $d_k = 0$, that F_k annihilates the code: it never happens on code states, so drop it.

- Both identities are the previous slide's equation divided by d_k: set $k = l$ for the first, $k \neq l$ for the second.
- Partial isometry: V_k preserves every inner product inside $\mathcal{C}$ and kills everything outside it. So on the code, F_k is $\sqrt{d_k}$ times a rigid move. The image $\mathcal{C}_k$ is an undistorted copy of $\mathcal{C}$, and different k give orthogonal copies.
- Set up the next slide: that geometry *is* what makes recovery possible. Orthogonal copies can be told apart by a measurement; undistorted copies can be rotated back.
- Closing remark on linearity: if R corrects $\{E_a\}$ it corrects any $F_r = \sum_a M_{ra} E_a$, because $R_k F_r P \propto P$ still. That is why we may restrict attention to a discrete error set.

- The previous slide built the geometry. This one runs the protocol on it.
- Starting point: $F_l P = \sqrt{d_l}\, V_l$, so the noisy state is a mixture. With probability d_l the code was moved rigidly onto copy l; we do not yet know which.
- Step 1: P_k is a projector because $V_k^\dagger V_k = P$ and $V_k P = V_k$, so $P_k^2 = V_k P V_k^\dagger = P_k$. Different copies are orthogonal because $V_k^\dagger V_l = 0$. Add $\mathbb{I} - \sum_k P_k$ to complete the measurement; that outcome never fires.
- Step 2: on a single branch, $P_k F_l \,|\psi\rangle = \delta_{kl} F_l \,|\psi\rangle$. So the outcome is certain and the state is not disturbed — it was already an eigenstate of the projector.
- The key point: $p_k = d_k$ does not depend on ρ. The measurement asks only a question whose answer is the same for every code state, so it learns the error and nothing about the logical information. That is the "α_{ab} independent of i" half of Knill–Laflamme, seen as a property of the measurement.
- Step 3: $V_k^\dagger$ is not unitary on the whole space. In the lab you apply any unitary that agrees with $V_k^\dagger$ on $\mathcal{C}_k$; one always exists because $\mathcal{C}_k$ and $\mathcal{C}$ have the same dimension.
- Order matters: measure first, then pick $V_k^\dagger$ (feedback). Averaged over outcomes, $\sum_k V_k^\dagger \varepsilon(\rho)\, V_k = \sum_k d_k\, \rho = \rho$, since completeness of the Kraus set gives $\sum_k d_k = 1$.
- Kitten preview: the copies are the even and odd parity sectors, so the syndrome is a parity measurement.
- Key sentence: the parity check we use in the lab is not an extra ingredient — it is the projective measurement that Knill–Laflamme forces into existence.

- One logical qubit in one cavity mode. Two codewords, two error words, five Fock levels in total — hence "kitten", a small sibling of the four-legged cat code.
- Parity is the organising principle. Codewords are even, loss images are odd, so a single parity measurement detects that a loss happened.
- Walk the arrows on the diagram. Note $\hat{a}\,|0\rangle = 0$, so only the $|4\rangle$ component of $|0_L\rangle$ survives the loss: $\hat{a}\,|0_L\rangle = (0 + 2\,|3\rangle)/\sqrt{2} = \sqrt{2}\,|3\rangle$.
- The $\sqrt{2}$ appearing in *both* lines is not a coincidence and it is not cosmetic. It is the whole reason this code works — next slide.
- If asked why $|4\rangle$ and not $|6\rangle$: binomial codes are built so the codewords' photon-number distributions match to the order of loss you want to correct. This one corrects single loss.

- Map the two facts onto the two halves of Knill–Laflamme explicitly — this is the payoff of having read the equation carefully earlier.
- Fact 1 is the δ_{ij} half. The images are orthogonal, so the loss is an isometry onto the error space; logical information is preserved, just relocated.
- Fact 2 is the "α independent of i" half. Both codewords have $\bar{n} = 2$: for $|0_L\rangle$ it is $(0+4)/2 = 2$, for $|1_L\rangle$ it is just 2. Photon loss is an amplitude-damping process whose *rate* depends on $\bar{n}$, so matched $\bar{n}$ means matched loss rate, which means the leak of information to the environment carries no logical content.
- Say the punchline: the syndrome is the parity flip, even $\to$ odd. Detectable and recoverable. This is how bosonic memories beat break-even while idling.
- Remember the number 2 — it comes back as $c_{aa} = 2$ in Condition 1.
- If asked about dephasing: what would break Fact 2 is the two codewords damping at different rates. Then the environment's record correlates with the logical bit and you get irreversible logical dephasing, which no end-of-gate recovery can undo. App. A raises exactly this as the reason Δ_{QEC} is worth minimising.

- This slide is the hinge of the talk. Do not rush it — the next equation is unmotivated without it.
- Idling QEC works because the codewords are nearly *static*. The photon leaves while the state sits in the configuration the code was designed around, so the loss image is one of two known error words.
- Say "nearly" on purpose. Kerr is on while idling too, so the error words pick up a small phase that depends on when the photon left. Our own check: with no drive at all, the trajectory-mismatch metric is already about 0.03. It is the same Kerr effect that comes back as the obstruction — just small when nothing moves.
- During a gate that premise is gone. At an intermediate time the state is not a codeword at all, just some vector the drive swept it to.
- Point at the figure: same ideal trajectory, three loss times, three endpoints. The final state is t-dependent, and t is a random variable we do not measure.
- So end-of-gate recovery has no single target to undo. A syndrome measurement can still tell you *that* a photon left, but not how to put the state back.
- Set up the next slide as the obvious fix: demand that the jump time stop mattering.

- Reframe the commuting square as a statement about the Hamiltonian. Moving the jump $\hat{a}$ from before H to after it produces the commutator $[H, \hat{a}]$. If that commutator is itself a correctable error, a loss at any time still ends as a correctable error at $t = T$. That is error closure (App. A, citing the error-closure literature); exact transparency, $[H, \hat{a}] = \lambda \hat{a}$, is the strict special case.
- Walk the table. The bare frequencies and the leading dispersive term give $\hat{a}$ back, times at most a transmon number operator: a deterministic phase on the error state (the paper's words).
- The linear drives: $[\epsilon \hat{a} + \epsilon^* \hat{a}^\dagger, \hat{a}] = -\epsilon^*$, a plain number, and the transmon drive commutes with $\hat{a}$. A number times the identity is in the correctable set $\{\mathbb{I}, \hat{a}\}$, so linear drives are fine. This is why the paper can use the standard cQED toolkit at all.
- Then the two that fail: self-Kerr K and the higher-order dispersive χ'. Their commutator is $\hat{n}\,\hat{a}$, a jump weighted by photon number — next slide.
- If pressed on the χ row: the phase depends on the transmon state, and mid-gate the transmon is driven. The paper counts it as correctable; the honest statement is "correctable as long as the transmon path is the planned one".
- Why defend linear drives: strong nonlinear drives have been proposed for exact ET, but they need strong nonlinearities and tight confinement to the static code space, and bring their own spurious processes. The paper's bet is that simpler wins.

- Recall the table on the last slide: the frequencies, the leading χ and the linear drives all pass the error-closure test. Only the nonlinear cavity terms fail.
- K' also fails, with a $-\frac{K'}{2}\hat{a}^{\dagger 2}\hat{a}^3$ term. At the photon numbers used it is much smaller than the K term (about 40 times at $n = 4$), and Eq. A6 leaves it out.
- The two that break it are the self-Kerr K and the higher-order dispersive χ'. Note for yourself: it is χ', the higher-order coupling, not the leading χ — and the sign in front of χ' is PLUS.
- Why $\hat{a}^\dagger\hat{a}^2$ is fatal: write it as $\hat{n}\,\hat{a}$. The $\hat{a}$ is the ordinary jump; $\hat{n}$ then weights the result by the photon number left behind. On the kitten, $|4\rangle \to |3\rangle$ gets weight 3 and $|2\rangle \to |1\rangle$ gets weight 1. Same error space, so the δ_{ij} demand still holds; but $\langle i_L|(\hat{n}\hat{a})^\dagger\hat{n}\hat{a}|i_L\rangle$ is 18 for $|0_L\rangle$ and 2 for $|1_L\rangle$, so the error learns which codeword it hit.
- Over a gate this shows up as a relative phase of about $2K(T-t)$ between the two error words: a logical Z rotation by a random angle, since t is random. Averaged over t, that is logical dephasing. Roughly 0.28 rad at most for a 1 μs gate.
- Numbers if asked: $K_a/2\pi = -22$ kHz for the cavity self-Kerr, $\chi/2\pi = -3.66$ MHz for the dispersive coupling.
- Wording: the paper says exact ET is "not expected" here, not that it is proven impossible. The commutator shows the operator-level route fails; say that, no more.
- Land the word: this is what "semi" in error semitransparent names. It is a deliberate trade, not a failure to converge. Keep repeating this — it explains why the metrics on trained pulses plateau at a floor.
- App. A's suggested escape routes, if asked: engineer smaller K and χ, keep intermediate photon number low, or go to strong nonlinear drives and pay the complexity cost.

- This is the paper's central conceptual move, and it is the reason the title says "dynamic encoding subspaces".
- The old framing fails for a trivial reason: a gate has to move the state, so insisting it stay inside a fixed two-dimensional subspace rules out the gates we want. Earlier exact-ET proposals accepted that and paid for it with strong nonlinear drives and very tight confinement.
- The fix: define the code space to be wherever the ideal evolution has carried the codewords by time t. By construction the ideal, lossless trajectory never leaves $\mathcal{C}(t)$.
- Emphasise that this is not a weaker demand in a hand-wavy sense — it is a different, well-posed demand. We still ask for consistency; we just ask for consistency with the intended trajectory rather than with a static subspace.
- Underline that $U(t,0)$ is the ideal propagator. All three metrics are computed from lossless unitary simulation; the loss operator appears only as the algebraic object E_j, never as an actual quantum jump.

- Same move, applied to the error space: push the error space along with the code space, so at each instant it is the image of the *current* code space under the error.
- For this code and this error the error space is again 2-dimensional — $\hat{a}$ restricted to the code space has rank 2. Worth flagging: in the repo's own implementation this is a real trap; feeding six cardinal states instead of the two code words yields four spurious basis vectors outside the error space and silently understates leakage.
- The proportionality in $P_{\mathcal{E}_j} \propto E_j P_{\mathcal{C}} E_j^\dagger$ is not an oversight. $E_j P_{\mathcal{C}}(t) E_j^\dagger$ is not a projector — $\hat{a}$ is not unitary, so it does not preserve norms. It has to be renormalised, and the paper fixes it only up to a constant. At $t = 0$ this is exact for the kitten code, because both $\hat{a}\,|0_L\rangle$ and $\hat{a}\,|1_L\rangle$ have norm $\sqrt{2}$ — the coincidence from the Knill–Laflamme slide. Mid-gate that coincidence fails by up to about 20%.
- Point at the figure: two tubes swept along the ideal trajectory. At any time slice you have a code 2-space and the error 2-space it maps onto under $\hat{a}$.
- Now we have the objects. The next four slides are the conditions that keep the commuting square closed on these moving subspaces.

- Give the audience the map before the algebra. Three conditions, and later exactly one scalar metric per condition — that pairing is the cleanest way to hold the rest of the talk in your head.
- One-line intuitions to say out loud:
- (1) is a *static* property checked at a moving location — freeze time, ask whether the code you are sitting on could still correct a photon loss.
- (2) is about consistency of *history* with *present* — an error that already happened and got carried forward must land in the same place as an error happening now.
- (3) is about *rates* — even if the two subspaces coincide, the drive must not rotate the state differently inside them.
- Note that (2) and (3) are genuinely independent: (2) is about which subspace you are in, (3) is about where inside it you are.

- Point out that this is literally the Knill–Laflamme equation from earlier, with $P_{\mathcal{C}} \to P_{\mathcal{C}}(t)$. Nothing new mathematically; the content is the t.
- The error set is $\{\mathbb{I}, \hat{a}\}$ — identity plus single photon loss — so there are four ordered pairs and hence four 2×2 blocks.
- Walk the matrix. $c_{II} = 1$: trivial, the code words stay orthonormal under unitary evolution, so that block is exactly the identity and contributes nothing. Off-diagonals vanish: $\langle 0_L|\hat{a}|1_L\rangle = 0$ because $\hat{a}\,|2\rangle \propto |1\rangle$ which is orthogonal to both code words, and likewise the other way. So the only live entry is c_{aa}.
- $c_{aa} = 2$ is the number from the kitten-code slide: $\bar{n} = 2$ for both code words. Call back to it explicitly — the audience saw it twenty minutes ago.
- Do not oversell the $t = 0$ picture. Matched $\bar{n}$ is only one component of one block. Mid-gate the drive mixes parities, so $\langle 0_L(t)|\hat{a}|1_L(t)\rangle$ and the off-diagonal parts of the $(\hat{a}, \hat{a})$ block need not vanish. Matched $\bar{n}$ is necessary, not sufficient.
- Why it can fail: photon loss is amplitude damping, and its rate scales with $\bar{n}$. If the drive pushes $|0_L\rangle$ to a higher $\bar{n}$ than $|1_L\rangle$, they damp at different rates, the environment's record correlates with the logical bit, and you get logical dephasing — irreversible, and not something the standard end-of-gate recovery can undo. App. A makes exactly this point.
- Presentational note to self: the paper's Eq. A7 defines an optional penalty $\Delta\bar{n}_L(t)$ that enforces just this. It is available but not part of the default objective — that comes up again on the cost-function overview.

- The equation is short; the content is in distinguishing the two objects. Spend the time there and use the tube picture from two slides ago.
- "Evolved" is a statement about history: the jump happened at $t = 0$ and the drive has been pushing the resulting state around ever since.
- "Instantaneous" is a statement about the present: if the jump happened right now, from the current code space, where would it put you?
- Condition 2 says these are the same subspace. If they are not, then whether a recovery works depends on when the jump happened — which is exactly the t-dependence we set out to eliminate.
- Anticipate the obvious objection, because someone will raise it: the equation only ever compares time t against time 0, but jumps happen at all times.
- Walk the block left to right. Eq. 4 at time s rewrites the space the jump lands in as an evolved copy of $\mathcal{E}_j(0)$. The two propagators then merge into one, $U(T, s)\,U(s, 0) = U(T, 0)$. Eq. 4 at time T turns that back into $\mathcal{E}_j(T)$.
- The punchline is that s drops out. Where the state ends up does not depend on when the jump happened, so the final recovery does not need to know it. $t = 0$ is a sufficient anchor because the propagator composes.
- Connect forward: this is the condition that the leakage metric $L_{\mathcal{E}_j}$ measures.

- Conditions 1 and 2 are about *which subspace* you occupy. Condition 3 is about *where inside it* you are. That distinction is the reason all three are needed.
- Even if the error space tracks perfectly, the drive could rotate the state within the error space differently from how it rotates the code state. Then the logical content of the two would drift apart, and the error would no longer commute through the gate.
- P_j is not a projector: it maps one space to another, like the undistorted-copy maps in the Knill–Laflamme sketch. So $P_j^\dagger H P_j$ is the error-space Hamiltonian pulled back onto the code space, which is what lets the two sides be compared.
- d_j is real: both sides of Eq. 5 are Hermitian, so their difference is too. An imaginary part would change the norm of the state, not just its phase.
- $d_j(t)\mathbb{I}$ is the allowed slack, and it is exactly the right amount of slack: a term proportional to the identity generates a global phase, which is invisible in ρ. Tie this back to $e^{i\phi_j(t)}$ on the commuting-square slide — it is the same freedom, expressed at the level of the generator instead of the state.
- Anything beyond that — any relative rotation — is a real, physical mismatch. That is what the trajectory-mismatch metric $\eta_{\mathcal{E}_j,\psi}$ will measure.
- If asked how this connects to the App. A obstruction: the uncorrectable $\hat{a}^\dagger \hat{a}^2$ commutator is precisely a term that fails Condition 3, because it generates a photon-number-dependent *relative* rotation, not a multiple of the identity.

- The pairing is the thing to sell here. Three conditions, three numbers, one-to-one.
- All three are computed from *lossless* unitary simulation. No quantum jumps are simulated; E_j enters only as an algebraic object. That is worth saying explicitly because it surprises people — these are properties of the ideal evolution, not of a stochastic trajectory.
- Time dependence matters: a pulse can look perfect at $t = T$ and be badly non-transparent in the middle, and the middle is exactly where a random photon loss might strike. This is why the cost function has to sum over the whole trajectory rather than just score the endpoint.
- Flag now, expand in the cost-function section: these are *diagnostics*. The optimiser's actual cost function is a different, differentiable object built from state overlaps.

- Structure the explanation as three moves: build the matrix, expand it, throw away the harmless part.
- Build: M_{ik} is Condition 1's left-hand side. Condition 1 wanted every M_{ik} proportional to $P_{\mathcal{C}}$; this metric measures the failure of exactly that.
- Expand: $P_{\mathcal{C}}(t)$ acts as the identity inside the 2-dimensional logical space, so $\{P_{\mathcal{C}}, X_{\mathcal{C}}, Y_{\mathcal{C}}, Z_{\mathcal{C}}\}$ is a complete basis and the expansion is unique.
- Strip: $\mathrm{Tr}[M_{ik}] = 2c_{ik}$ because the logical Paulis are traceless and $\mathrm{Tr}\, P_{\mathcal{C}} = 2$. So subtracting $\frac{1}{2}\,\mathrm{Tr}[M_{ik}]P_{\mathcal{C}}$ removes precisely the correctable piece, leaving only uncorrectable generators.
- Note to self on the norm — this is the one place my own notes are wrong. I wrote $\|A\|_2 = \sqrt{\lambda_{\max}(A^\dagger A)}$, the spectral norm, but then quoted the answer $2(|x|^2 + |y|^2 + |z|^2)$, which is the Frobenius result. The main text says "2-norm" but sends you to App. A, and App. A defines $\|A\|_2 = \mathrm{Tr}(A^\dagger A)$ with no square root. Frobenius is the right reading, and it is what our own implementation uses.
- The $(\mathbb{I}, \mathbb{I})$ pair contributes exactly zero: $M_{II} = P_{\mathcal{C}}$ is pure identity, because unitary evolution keeps the code words orthonormal.
- Not normalised by anything — so absolute values are directly comparable to the paper's.

- Unpack why the maximally mixed state is the right initial condition: it is proportional to the projector, so $\mathrm{Tr}[P_{\mathcal{E}_j}(t)\rho(t)]$ is an average over all logical inputs rather than a statement about one particular state. That is what makes this a property of the *subspace* and not of a chosen vector.
- For the kitten code, $P_{\mathcal{E}_j}(0) = |1\rangle\langle 1| + |3\rangle\langle 3|$, so $\rho(0) = \frac{1}{2}(|1\rangle\langle 1| + |3\rangle\langle 3|)$.
- The key identity that makes this a subspace-overlap: for any state, $\mathrm{Tr}[P\,|\phi\rangle\langle\phi|] = \langle\phi|P|\phi\rangle$. So the trace expands into the mean of $\left\langle 0_E(t)\middle|P_{\mathcal{E}_j}(t)\middle|0_E(t)\right\rangle$ and the same for $|1_E(t)\rangle$ — the alignment of the two error subspaces.
- Emphasise the evolved-versus-instantaneous framing; it is the same distinction as Condition 2 and this is its scalar.
- Worth stating: no loss is ever applied during the evolution. The error is applied once, at $t = 0$, and then everything is unitary.
- If asked how this differs from $\eta_{\mathcal{E}_j,\psi}$: leakage asks whether you are in the right *subspace*; $\eta_{\mathcal{E}_j,\psi}$ asks, given that you are, whether you are at the right *place inside it*. $\eta_{\mathcal{E}_j,\psi}$ projects the error state into the instantaneous error space first, which makes it deliberately blind to leakage. So the two carry genuinely independent information, and neither bounds the other.

- The picture to describe: two Bloch spheres side by side, one for the code space and one for the error space, each carried along by the drive. Put a state on the code sphere and its error partner on the error sphere. At $t = 0$ they sit at the same point. Transparency says they must stay at the same point.
- Stress "its own instantaneous frame". The logical Pauli operators are themselves being carried along by the drive; we are not comparing against a fixed lab frame. That is what makes this a statement about *relative* rotation.
- Why a global phase does not show up: Bloch vectors come from ρ, and ρ is insensitive to global phase. So the $d_j(t)\mathbb{I}$ freedom of Condition 3 is automatically forgiven by this metric — which is exactly what we want.
- Six cardinal points, $\pm X$, $\pm Y$, $\pm Z$, give six code/error pairs. Testing a single direction would not pin the map: two basis states fix a complex 2-dimensional map only up to relative phases, and all six fix it completely.
- Range is $[0, 2]$ — the diameter of the Bloch sphere — so the numbers are easy to read: 0.2 is a modest rotation, 1.0 is severe.
- One interpretive wrinkle if anyone presses: the paper fixes $\vec{\sigma}_{\mathcal{E}_j}$ only up to a proportionality constant, because $E_j P_{\mathcal{C}} E_j^\dagger$ is not a projector. Our implementation resolves it with the polar isometry of $\hat{a} P_{\mathcal{C}}(t)$; the sensitivity to that choice is about 0.4%.

- The key term is χ. Written as $(\omega_a + \chi\, \hat{q}^\dagger \hat{q})\hat{a}^\dagger \hat{a}$ it is a cavity frequency that depends on the transmon; written as $(\omega_q + \chi\, \hat{a}^\dagger \hat{a})\hat{q}^\dagger \hat{q}$ it is a transmon frequency that depends on photon number. $|\chi|/2\pi = 3.66$ MHz is far larger than the transmon linewidth, so the photon-number peaks are resolved: a transmon pulse longer than $\sim 1/|\chi|$ (≈ 270 ns) can address individual photon numbers.
- The Kerr-type terms are tiny ($K/2\pi = -22$ kHz), but they do not commute nicely with $\hat{a}$. Plant the seed: this is where "semi" will come from.
- Rotating frame: in the simulation there is no ω_a or ω_q. What remains is the static nonlinearity the pulse has to work with and against.
- Photon loss, jump operator $\hat{a}$, is the error the kitten code is designed to correct. It is the E_j in the Kraus form of the setup slide.
- Flag the symbol clash out loud: drive $\epsilon(t)$ versus channel $\varepsilon(\rho)$.